The Journal of Open Source Software

couplr: Optimal pairing and matching via linear assignment

Gilles Colling [1¶]

1 Department of Botany and Biodiversity Research, University of Vienna, Austria ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

`couplr` (Colling, 2026) is an R package for optimal pairing, matching, and assignment. It solves linear assignment problems: given two sets of objects and a cost for each possible pair, choose the pairs with minimum total cost. That operation appears in causal inference, experimental design, task allocation, sample pairing, and image alignment.

The package provides two entry points. A high-level data-frame interface prepares data, builds distances, enforces constraints, and returns analysis-ready matched output for optimal, greedy, propensity-score, full, coarsened-exact, subclassification, and cardinality matching, with balance diagnostics and Rosenbaum sensitivity bounds (Rosenbaum, 2002). A low-level interface exposes the solvers directly when the user already has a cost matrix. An automatic dispatcher routes each problem to one of 18 internal solvers based on shape, sparsity, cost type, and size, so the same package supports statistical matching workflows and general-purpose assignment tasks.

Statement of need

A serious matching workflow in R today threads three packages by hand: one for the assignment solver, one for preprocessing and constraint construction, one for balance diagnostics. Any non-default choice (exact blocking, calipers on multiple variables, pairwise distances on observed covariates, variable-ratio full matching, replacement matching, direct control over the solver) pulls the user into a different corner of that toolchain.

`couplr` packages the workflow around the linear assignment problem. A causal-inference user starts with a data frame, observed covariates, and a treatment indicator; an operations-research user starts with a cost matrix already in hand. The same solver core serves both.

State of the field

`MatchIt` (Ho et al., 2011), `optmatch` (Hansen & Klopfer, 2006), `Matching` (Sekhon, 2011), and `designmatch` (Zubizarreta et al., 2024) handle preprocessing and matching for causal inference; `cobalt` supplies cross-package balance diagnostics (Greifer, 2025). For the assignment problem itself, R users reach for `clue` (Hungarian via `solve_LSAP`) or `lpSolve` (general LP) (Berkelaar & others, 2024; Hornik, 2024). The causal-inference packages assume a treated/control framing and route through a single fixed back-end (`optmatch` for `MatchIt`, network flow for `optmatch`, genetic search for `Matching`, integer programming via commercial solvers for `designmatch`); the solver-focused packages expose one algorithm each without the preprocessing, constraint, or diagnostic machinery a matching workflow needs.

`couplr` combines both. It treats covariate distance as the primary entry point rather than a side option behind a propensity score, and ships 18 assignment solvers spanning classical dense assignment, sparse and rectangular variants, k-best (Murty, 1968), bottleneck, min-cost flow, and entropy-regularized transport, with an automatic dispatcher.

Software design

The package has three layers. The first validates inputs and converts data frames into cost matrices, handling scaling, missing-data checks, blocking, calipers, maximum-distance rules, and user weights. The second solves the linear assignment problem: given a cost matrix $C \in \mathbb{R}^{n\times m}$ with entries C_{ij} for row $i \in \{1, \dots, n\}$ and column $j \in \{1, \dots, m\}$, find a binary assignment matrix $X \in \{0, 1\}^{n\times m}$ ($X_{ij} = 1$ iff row i is matched to column j) solving

$$\min_X \sum_{i,j} C_{ij}X_{ij} \quad \text{subject to} \quad \sum_j X_{ij} \leq 1 \ \forall i, \quad \sum_i X_{ij} \leq 1 \ \forall j.$$

The bipartite structure makes the LP relaxation integral (Birkhoff–von Neumann), so row potentials $u_i \in \mathbb{R}$ ($i = 1, \dots, n$) and column potentials $v_j \in \mathbb{R}$ ($j = 1, \dots, m$) satisfying

$$u_i + v_j \leq C_{ij} \quad \forall (i, j), \qquad u_i + v_j = C_{ij} \quad \text{whenever } X_{ij} = 1$$

certify optimality; `assignment_duals()` returns (u, v). Worst-case time is $O(n^3)$ for Hungarian and Jonker–Volgenant, $O(n^3 \log(n\,C_{\max}))$ for cost scaling, and $O(n^{3/4} m \log(n\,C_{\max}))$ for Gabow–Tarjan bit scaling, where $C_{\max} = \max_{i,j} |C_{ij}|$ is the largest cost magnitude; the dispatcher selects among solvers using n, m, sparsity, and rectangularity. All 18 solvers are implemented from scratch in C++ via Rcpp and RcppEigen, so the package adds no external solver dependency: the Hungarian method (Kuhn, 1955), Jonker–Volgenant shortest augmenting paths (Jonker & Volgenant, 1987), auction algorithms (Bertsekas, 1988), cost scaling (Goldberg & Kennedy, 1995), Gabow–Tarjan scaling (Gabow & Tarjan, 1989), push-relabel min-cost flow ideas (Goldberg & Tarjan, 1988), network simplex, Ramshaw–Tarjan rectangular assignment (Ramshaw & Tarjan, 2012), and related specialized solvers. The third layer returns structured result objects with pairs, costs, diagnostics, weights, subclass information, and conversion methods for other R matching tools.

Distances cover Euclidean, Manhattan, Mahalanobis (pooled within-group covariance by default), Chebyshev, and squared-Euclidean metrics, or a user-supplied function; propensity-score distances use a separate entry point. Three constraint mechanisms reshape the cost matrix before it reaches the solver: a global `max_distance` ceiling, per-variable calipers, and forbidden pairs encoded as large finite costs so the matrix stays usable across all solvers. Blocking either reuses a factor variable or constructs blocks via k-means or hierarchical clustering, and the solver is called once per block.

The dispatcher in `lap_solve(method = "auto")` chooses by matrix shape, sparsity, cost type, and size: dense square problems use Jonker–Volgenant or cost scaling; sparse matrices use sparse augmenting-path solvers; rectangular problems avoid padding through rectangular assignment methods. Figure 1 shows median solve time across problem sizes for all 18 solvers; benchmarks are reproducible via `paper/make-figure.R`.

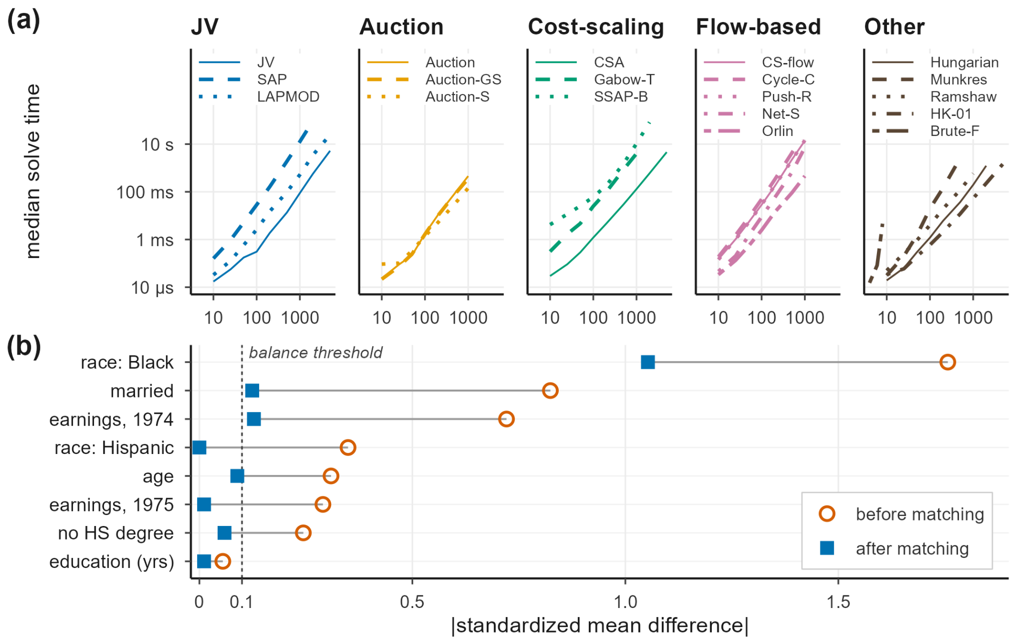

Figure 1: (a) Median wall-clock solve time versus problem size n for all 18 assignment solvers in `couplr`, arranged as five small-multiple panels grouped by algorithm family (JV / augmenting-path, Auction, Cost-scaling, Flow-based, and Other). Within each panel, individual solvers share a single family colour and are distinguished by line style; all panels share log–log axes. Median of 5 replicates on a single core; integer cost matrices with entries in $[1, 10{,}000]$. (b) Absolute standardized mean differences on the LaLonde NSW data (185 treated, 429 control, eight covariates) before and after 1-to-1 optimal Mahalanobis matching. After matching, `couplr`, `MatchIt`, and `optmatch` produce identical $|SMD|$ to three decimal places, so a single matched point per covariate is shown. Seven of the eight covariates fall below the 0.1 balance threshold (dashed line); the `race_Black` indicator is reduced from 1.757 to 1.053, a residual no 1-to-1 matcher can resolve here. Reproducible from `paper/make-figure.R` and `paper/bench_lalonde.R`.

`balance_diagnostics()` reports standardized mean differences, variance ratios, and Kolmogorov–Smirnov statistics, summarised through `balance_table()`; `sensitivity_analysis()` reports the critical Rosenbaum Γ at which inference would no longer be significant (Rosenbaum, 2002). Blocked designs surface per-block diagnostics so imbalance is not hidden by averaging across strata.

Quickstart

The package ships with `hospital_staff`, a small example dataset for a treated/control workflow that runs without external data.

```
library(couplr)

data(hospital_staff)
treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)

# Optimal one-to-one matching on three pre-treatment covariates, with
# automatic scaling and solver selection.
m <- match_couples(
  left  = treated,
```

The Journal of Open Source Software

```r
  right = control,
  vars  = c("age", "experience_years", "certification_level"),
  auto_scale = TRUE
)

# Balance on matching variables; analysis-ready paired output.
bal <- balance_diagnostics(m, treated, control,
  vars = c("age", "experience_years", "certification_level"))
balance_table(bal)
matched <- join_matched(m, treated, control)
```

On this dataset the matched pairs achieve absolute standardized differences below 0.15 on all three matching covariates. The same `match_couples()` call accepts `calipers`, `max_distance`, `block_id`, `method`, `replace`, and `ratio` for harder problems. Identification also requires conditional ignorability, positivity, and SUTVA; the matched output is designed to feed into an outcome regression for a doubly robust estimate (Stuart, 2010).

Comparison against established alternatives

We compare `couplr` against `MatchIt` and `optmatch` on runtime and feature coverage. The matching task is fixed: 1-to-1 optimal matching on a Mahalanobis distance with pooled within-group covariance, the convention `optmatch::match_on()` uses by default and `couplr` adopts as of 1.3.1. On the canonical Lalonde NSW dataset (Dehejia & Wahba, 1999; LaLonde, 1986) (185 treated, 429 control, eight covariates) all three packages produce identical absolute standardized mean differences to three decimal places, reducing the maximum $|\text{SMD}|$ from 1.757 to 1.053 on `race_Black` (a residual no 1-to-1 matcher can resolve here) and the remaining seven covariates to ≤ 0.124 (Stuart, 2010); per-covariate values are in `paper/lalonde-per-covariate.csv`.

Table 1 reports median wall-clock time on synthetic problems with the same eight-covariate structure, at five sizes from $n = 500$ to $n = 20{,}000$ with treated:control $= 1{:}2$. `couplr` is 1.5–$1.6\times$ faster than `optmatch` and 2.0–$2.5\times$ faster than `MatchIt` where all three complete; at $n = 20{,}000$ `couplr` finishes in 3.5 minutes, `optmatch` in 11 minutes ($3.1\times$ slower), and `MatchIt::matchit(method = "optimal")` aborts inside the `optmatch` backend with an integer-size overflow. `couplr` reaches large dense problems through its dispatcher, which routes to Jonker–Volgenant (Jonker & Volgenant, 1987), auction with scaling (Bertsekas, 1988), and cost-scaling (Goldberg & Kennedy, 1995) algorithms.

Table 1: 1-to-1 optimal Mahalanobis matching, median wall-clock by problem size. Treated:control $= 1{:}2$; eight covariates; pooled within-group covariance; single core, single-threaded BLAS. Median of 5 / 5 / 3 / 3 / 1 replicates respectively for the rows; `optmatch_max_problem_size` set to `Inf` for $n \geq 10{,}000$. `int overflow` marks an integer-size overflow (`result would exceed 2^31-1 bytes`) inside the `optmatch` backend reached through `MatchIt`. Reproducible from `paper/bench_scaling.R` and `paper/bench_scaling_alternatives.R`.

Problem size ($n_t + n_c$)	`couplr`	`optmatch`	`MatchIt`
$167 + 333$	80 ms	120 ms	190 ms
$667 + 1{,}333$	1.37 s	2.09 s	3.40 s
$1{,}667 + 3{,}333$	10.1 s	16.4 s	23.8 s
$3{,}333 + 6{,}667$	53.7 s	79.3 s	110 s
$6{,}667 + 13{,}333$	210 s	657 s	int overflow

Table 2 summarises feature coverage. `couplr` exposes 18 solvers through a single dispatcher, supports k-best, bottleneck, and Rosenbaum sensitivity bounds in the core package, and accepts both data frames and user-supplied cost matrices.

Table 2: Differentiating feature coverage; rows where all three packages support the feature (covariate distance, propensity-score matching, exact blocking, cost-matrix entry point) are omitted. *partial* indicates the feature is achievable but not via a first-class user-facing API.

Feature	`couplr`	`MatchIt`	`optmatch`
Per-variable calipers (named vector)	yes	yes	partial
Rectangular $n_t \neq n_c$, no padding	yes	partial	partial
Sparse-cost support	yes	no	yes
k-best assignments (Murty)	yes	no	no
Bottleneck (minimax) assignment	yes	no	no
Rosenbaum sensitivity bounds	yes	no	no

AI usage disclosure

The package was developed using a modern AI-assisted developer stack. The author designed the package architecture, made the algorithm-selection choices, and wrote the implementation in a TUI-first command-line workflow, using a self-hosted Qwen3-Coder-Next-REAP-48B-A3B model (mixture-of-experts coder, ~3B active per token, 4-bit MLX quantization, running on a single Apple M4 Pro) for code completion, refactoring, and boilerplate generation through Anthropic's Claude Code CLI, with requests routed to the local model and no cloud inference.

Acknowledgements

No external funding supported this work.

References

Berkelaar, M., & others. (2024). *lpSolve: Interface to lp_solve v. 5.5 to solve linear/integer programs*. https://CRAN.R-project.org/package=lpSolve

Bertsekas, D. P. (1988). The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, *14*, 105–123. https://doi.org/10.1007/BF02186476

Colling, G. (2026). *Couplr: Optimal pairing and matching via linear assignment*. https://doi.org/10.32614/CRAN.package.couplr

Dehejia, R. H., & Wahba, S. (1999). Causal effects in nonexperimental studies: Reevaluating the evaluation of training programs. *Journal of the American Statistical Association*, *94*(448), 1053–1062. https://doi.org/10.1080/01621459.1999.10473858

Gabow, H. N., & Tarjan, R. E. (1989). Faster scaling algorithms for network problems. *SIAM Journal on Computing*, *18*(5), 1013–1036. https://doi.org/10.1137/0218069

Goldberg, A. V., & Kennedy, R. (1995). An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, *71*(2), 153–177. https://doi.org/10.1007/BF01585996

Goldberg, A. V., & Tarjan, R. E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, *35*(4), 921–940. https://doi.org/10.1145/48014.61051

Greifer, N. (2025). cobalt: Covariate balance tables and plots. *Journal of Open Source Software*, *10*(109), 7610. https://doi.org/10.21105/joss.07610

Hansen, B. B., & Klopfer, S. O. (2006). Optmatch: Flexible, optimal matching for observational studies. *The R Journal*, *2*(2), 29–36. https://journal.r-project.org/archive/2006-2/

RJournal_2006-2_Hansen+Klopfer.pdf

Ho, D. E., Imai, K., King, G., & Stuart, E. A. (2011). MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, *42*(8), 1–28. https://doi.org/10.18637/jss.v042.i08

Hornik, K. (2024). *Clue: Cluster ensembles*. https://CRAN.R-project.org/package=clue

Jonker, R., & Volgenant, T. (1987). A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, *38*(4), 325–340. https://doi.org/10.1007/BF02278710

Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1-2), 83–97. https://doi.org/10.1002/nav.3800020109

LaLonde, R. J. (1986). Evaluating the econometric evaluations of training programs with experimental data. *The American Economic Review*, *76*(4), 604–620. https://www.jstor.org/stable/1806062

Murty, K. G. (1968). An algorithm for ranking all the assignments in order of increasing cost. *Operations Research*, *16*(3), 682–687. https://doi.org/10.1287/opre.16.3.682

Ramshaw, L., & Tarjan, R. E. (2012). *On minimum-cost assignments in unbalanced bipartite graphs* (HPL-2012-40R1). HP Laboratories. https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html

Rosenbaum, P. R. (2002). *Observational studies* (2nd ed.). Springer. https://doi.org/10.1007/978-1-4757-3692-2

Sekhon, J. S. (2011). Multivariate and propensity score matching software with automated balance optimization: The Matching package for R. *Journal of Statistical Software*, *42*(7), 1–52. https://doi.org/10.18637/jss.v042.i07

Stuart, E. A. (2010). Matching methods for causal inference: A review and a look forward. *Statistical Science*, *25*(1), 1–21. https://doi.org/10.1214/09-STS313

Zubizarreta, J. R., Kilcioglu, C., & Vielma, J. P. (2024). *Designmatch: Matched samples that are balanced and representative by design*. https://CRAN.R-project.org/package=designmatch